

- ✓ Blocs PL/SQL (suite)
- ✓ Exception
- ✓ Curseur

Introduction : Dans ce TP, nous allons continuer de pratiquer la programmation en PL/SQL en exploitant les notions vues dans le cours à savoir le bloc PL/SQL anonyme, les exceptions et les Curseurs

Exercice I- Bloc PL/SQL et Exceptions

Soit la table des Adhérents suivante ADHERENTS (**id_adherent**, nom, prenom , date_naissance, fonction)

id_adherent	prenom,	nom ,	date_naissance,	fonction
100	BADR	DOLIN	12/09/1980	Ingénieur
102	ASSOU	TAHA	02/08/2003	
111	HAMID	TAHA	17/02/2002	Lycien
222	ALI	FAN	12/09/1980	Professeur
123	KHALIL	IBN SINA	12/09/1980	Technicien
210	YASSER	SOFFI	12/09/1994	Etudiant
214	RIMI	FAN	30/03/2000	

1. Créer la table ADHERENTS et insérer ces enregistrements
2. Écrire un bloc PL/SQL qui demande à l'utilisateur l'**id_adherent** de l'adhérent puis il affiche son nom et son prénom et sa date de naissance,
3. Refaire le même programme en introduisant les deux exceptions suivantes
 - La 1^{ère} affiche le message suivant si le **id_adherent** n'existe pas (SVP vérifiez vos informations, cet ID n'existe pas)
 - La 2^{em} affiche le message suivant si le l'adhérent correspondant n'a pas de fonction (SVP vérifiez vos information, L'adhérent nom prénom n'a pas de fonction)
4. Écrire un bloc PL/SQL qui demande à l'utilisateur un Nom de l'adhérent puis il affiche : son nom, son prénom et sa fonction,
 - 4.1 Exécuter le programme en entrant le nom « **FAN** » et le nom « **TOBKAL** »
 - 4.2 Qu'ils sont les exceptions qu'on peut traiter à part. ?
 - 4.3 Refaire les programmes en prenant en compte ces exceptions

Exercice II- Curseurs

- 1- Le code ci-dessous est conçu pour afficher les employés ayant un salaire inférieur à 3000. Vous taper ce code directement. Le but est de regarder ce que vous obtenez à la sortie, et être en mesure de décrire ce que fait chaque ligne de code.

```
DECLARE
CURSOR bas_salaires IS SELECT last_name, first_name,salary FROM employees where salary <3000;
v_snum          employees.last_name%TYPE;
v_surname       employees.first_name%TYPE;
v_salary        employees.salary%TYPE;
BEGIN
OPEN bas_salaires;
LOOP
    FETCH bas_salaires INTO v_snum,v_surname,v_salary;
    EXIT when bas_salaires%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE(v_snum || ' ' ||v_surname|| ' ' || v_salary);
END LOOP;
CLOSE bas_salaires;
END;
```

Question : Après avoir taper ce code vous décrivez ce que fait chaque ligne.

2- **Code avec erreur !**

Le code du curseur ci-dessous est conçu pour mettre à jour les salaires des employés en fonction de l'endroit où ils travaillent, mais il contient beaucoup d'erreur. Regarder très, très attentivement et essayer d'apprendre en interprétant les messages d'erreur que vous obtenez. Parfois, ils sont très utiles.

Important

Avant de taper ce code et pour ne pas toucher à la table **EMPLOYEES** directement, je vous invite à en créer une copie dans la table **EMPLOYEE_TMP** par la commande : **create table EMPLOYEE_TMP as select * from EMPLOYEES**

```
declare
cursor c1 select employee_id, city from employees e, departments d,locations l
where d.location_id=l.location_id and e.department_id=d.department_id;
v_city      l.city%type ;
v_snum      employees.employee_id%type;
v_salary_1  number:=30000; --2
v_salary_2  number(4):=25000;
v_salary_3  number:=20000;
v_salary    employees.salary%type;
begin
open c1;
loop
    fetch c1 into v_snum, v_city;
```

```

        exit where c1%NOTFOUND;
    If v_city='Oxford' then v_salary:=v_salary_1;--v_salary1;
        elsif v_city='Seattle' then v_salary:=v_salary_2;
        else v_sal:=v_salary_3;
        end if;
        update employe_tmp set salary = v_salary where employee_id=v_snum;
    endloop;
    close ;
end;
```

Question : Quand vous corrigez toutes les erreurs vérifier est ce que la mise à jour est bien effectuée dans la table **EMPLOYE_TMP**

- 3-** Créer la table REVMAX (noemp number(4), nom varchar(10), revenu number(10,2)); et créer un bloc PL/SQL qui insère dans cette table les cinq premiers employés qui ont les revenus les plus élevés. Vérifier l'insertion

NB : - **revenu = salary (1+ commission_PCT) ;** avec **commission_PCT** est la commission de l'employée

- Faire attention dans le cas ou la "**commission_PCT**" est null, utiliser la fonction **nvl** :

revenu = salary *(1+ nvl(commission_PCT,0))

- 4-** En utilisant la table ADHERENTS, réaliser un programme PL/SQL, qui vérifie la date de naissance de chaque adhérent et nous informe de la nature de l'adhérent (JUNIOR, ou SENIOR) par rapport à la date d'aujourd'hui (Si l'âge de l'adhérent >18ans c'est SENIOR sinon c'est JUNIOR)

Le message sera comme suite :

BADR DODIN est SENIOR

ASSOU TAHA est JUNIOR

.....etc

NB : Utiliser un **curseur** et la fonction **to_date** pour faire la différence des dates

(to_date(sysdate,'DD/MM/YY') - to_date(variable_date_de_naissance,'DD/MM/YY'))/365

- 5-** En utilisant un Curseur, Écrivez un programme PL/SQL qui calcule la moyenne des salaires des employés dont la date de recrutement est entre 01/01/2002 et 01/01/2005.

Comparer avec la requête ci-dessous, vous devez trouver la même chose

(SELECT avg(salary) FROM employees

WHERE (to_date(hire_date,'DD/MM/YY') between '01/01/02' and '01/01/05');

- 6- Refaire la question 5 en passant les deux dates 01/01/2002 et 01/01/2005. En paramètre de curseur
- 7- Écrivez un programme PL/SQL qui permet d'afficher les enregistrements (employee_id , Nom et Salaire) de la table EMPLOYEES qui ont un salaire supérieur à la moyenne des salaires des employés en utilisant la **boucle FOR pour parcourir le curseur**

NB : Utiliser le chr(9) comme code ascii de la tabulation entre les champs lors de l'affichage par dbms_output.put_line **Exemple :** (dbms_output.put_line('NO: ' || curseur.employee_id || chr(9) || 'NOM: ' || curseur.first_name || chr(9) || ' SALAIRE:' || curseur.salary);